SIMATS ENGINEERING
ASSESSMENT TOOL 1
Annexure A – Design Task

Course Name	Artificial Intelligence
Course Code	CSA17
Course Outcome	CO5
Description	Design AI-based systems using PySpark, RDD operations, and intelligent agent
Assessment Weightage	50%
Duration	1 Hour
Topic Chosen	Smart Disaster Detection System

Submitted by

Name: S. KARTHIKEYAN

Register No: 192419048

Code of Conduct

I, S. Karthikeyan (Reg. No: 192419048), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Smart Disaster Detection System – Distributed AI Design

Problem Statement: Develop a distributed AI system using PySpark and RDD operations to detect natural disasters (floods, earthquakes) from multi-source data streams (satellite images, sensors, and social media), describing the system architecture, data fusion techniques, and how intelligent agents coordinate to provide early warnings and response strategies.

1. Problem Understanding & Requirement Analysis

Natural disasters such as floods and earthquakes must be detected within seconds to minutes of onset for early warnings to be useful, yet the evidence for an unfolding disaster is scattered across heterogeneous, high-volume sources: satellite/aerial imagery (large, slow-arriving, coarse temporal resolution), ground sensors such as seismographs and river-gauge/water-level sensors (small, very high-frequency, structured), and social media posts (huge volume, unstructured text/images, noisy but near real-time). The core design challenge is therefore to **ingest, fuse, and reason over these three very different data modalities at scale and in near real time**, and to convert the resulting disaster assessment into an actionable, geographically targeted warning.

Key requirements: (i) horizontal scalability to handle bursty, high-volume streams during an actual disaster event; (ii) fault tolerance, since losing data during a disaster is unacceptable; (iii) low end-to-end latency from raw signal to warning; (iv) the ability to combine evidence of different reliability and modality (sensor fusion); and (v) an architecture that can act autonomously (via agents) once a threshold of confidence is reached, rather than waiting for a human operator to manually correlate every source.

2. System Architecture Design

The system is organised into five layers:

- **Data Ingestion Layer:** Kafka/Flume-style streaming connectors collect satellite image tiles (periodic batch feed), sensor telemetry (continuous high-frequency stream from IoT/seismograph/river-gauge networks), and social media posts (filtered by disaster-related keywords/hashtags and geotags) into a message queue that feeds Spark Streaming.
- **Distributed Processing Layer (PySpark/RDD core):** Spark Streaming consumes the queued data as micro-batches of RDDs; separate RDD pipelines pre-process each modality in parallel across the cluster (image feature extraction, sensor signal filtering, text/NLP cleaning).
- **Data Fusion & Detection Layer:** Per-modality RDDs are joined/co-grouped by geographic grid cell and time window to produce a fused feature vector per region, which is scored by anomaly/event-detection models to yield a region-level disaster-likelihood score.
- **Intelligent Agent Layer:** Cooperating software agents (Sensor Agents, a Fusion/Coordinator Agent, and Response Agents) monitor the fused scores, negotiate/confirm whether a genuine event is occurring, and trigger warnings.
- **Alerting & Response Layer:** Confirmed alerts are pushed to a dashboard, SMS/public-alert gateway, and emergency-response coordination systems, geographically targeted to the affected grid cells.

3. Use of PySpark & RDD Operations

Each modality is represented as its own RDD, keyed by a (region_id, time_window) tuple so that they can later be combined:

```
# Satellite image RDD: extract flood-extent / thermal-anomaly features per tile
sat_rdd = sat_stream.map(lambda tile: (tile.region_id,
                                       extract_image_features(tile)))

# Sensor RDD: seismograph + river-gauge readings
sensor_rdd = sensor_stream.map(lambda r: (r.region_id,
                                          (r.timestamp, r.type, r.value)))
sensor_rdd = sensor_rdd.filter(lambda kv: is_within_threshold_window(kv))

# Social media RDD: geotagged posts filtered for disaster keywords
social_rdd = social_stream.filter(lambda p: contains_disaster_keywords(p.text))
social_rdd = social_rdd.map(lambda p: (p.region_id, nlp_sentiment_urgency(p.text)))

# Aggregate each modality per region (reduceByKey = distributed, parallel)
sat_agg    = sat_rdd.reduceByKey(lambda a, b: merge_image_features(a, b))
sensor_agg = sensor_rdd.groupByKey().mapValues(compute_sensor_anomaly_score)
social_agg = social_rdd.groupByKey().mapValues(compute_social_urgency_score)

# Fuse all three modalities per region via join
fused = sat_agg.join(sensor_agg).join(social_agg) \
               .mapValues(lambda v: fuse_scores(v))   # -> region disaster score

# Flag high-risk regions for the agent layer
alerts = fused.filter(lambda kv: kv[1] > ALERT_THRESHOLD).collect()
```

This design uses **map** and **filter** for parallel, per-record pre-processing of each modality; **reduceByKey/groupByKey** to aggregate many individual sensor readings or posts into one score per region (distributed across executors, avoiding a single-node bottleneck); **join** to perform the multi-modal data fusion itself, since it combines RDDs keyed on the same region identifier; and a final **filter + collect** action to bring only the small set of high-risk alerts back to the driver for the agent layer to act on, keeping the expensive computation distributed while keeping the final decision step lightweight.

4. Intelligent Agent Integration

Three cooperating agent types coordinate the response, following a belief-desire-intention (BDI)-style architecture:

- **Sensor/Source Agents** (one per modality/region): continuously monitor their RDD-derived scores and raise a local belief ("possible flood in region X") when their own score crosses a modality-specific threshold.
- **Fusion/Coordinator Agent:** collects beliefs from all Sensor Agents for a region, weighs them by source reliability (e.g., seismograph readings weighted higher than unverified social posts), and decides whether the combined evidence meets the confidence level required to declare an event.
- **Response Agents:** once the Coordinator Agent confirms an event, these agents select and dispatch the appropriate response strategy (public SMS alert, evacuation-route notification, emergency-service dispatch) and continue to monitor the region for escalation or resolution, feeding outcomes back to improve future threshold calibration.

This multi-agent design lets the system react in real time even as raw data keeps streaming in: agents operate on the continuously updated fused RDD scores rather than waiting for a full batch job to finish, and the negotiation step between Sensor Agents and the Coordinator Agent reduces false alarms that a single-source system would otherwise generate (e.g., a spike in social-media chatter alone is not treated as sufficient evidence of an actual earthquake).

5. Scalability & Distributed Computing Design

Scalability is addressed at every layer: the ingestion layer uses partitioned message queues so throughput scales horizontally with the number of consumer partitions; Spark's RDD lineage graph provides **fault tolerance** by allowing lost partitions to be recomputed from lineage rather than requiring full-job restarts, which is critical during a disaster when nodes may fail; **data locality-aware scheduling** keeps sensor/image processing close to where the data lands, reducing network overhead; and the pipeline is **elastic** — additional executors can be added dynamically as data volume spikes during an actual unfolding event, which is precisely when load is highest and dropped data is least acceptable. Checkpointing of streaming state further ensures that the fusion layer can recover its in-progress region aggregates after a driver or executor failure.

6. Data Flow & Processing Pipeline

The end-to-end flow is: **raw multi-source streams** → **ingestion queue** → **Spark Streaming micro-batches (RDDs)** → **per-modality cleaning/feature extraction (map/filter)** → **per-region aggregation (reduceByKey/groupByKey)** → **cross-modality fusion (join)** → **threshold filtering** → **Sensor Agent beliefs** → **Coordinator Agent confirmation** → **Response Agent action** → **dashboard/alert dispatch**, with a feedback loop from confirmed/resolved events back into threshold calibration for future detection cycles. This pipeline is deliberately structured so that each stage's output is a smaller, more distilled RDD than its input, progressively reducing data volume as information moves from raw signals toward an actionable decision.

7. Innovation & Creativity

The design's distinctive elements are: (i) treating source reliability as a tunable weight in the Fusion Agent rather than trusting all modalities equally, which directly addresses the well-known unreliability of social-media signals; (ii) a feedback loop where confirmed/false alarms retrain the fusion thresholds over time, so the system's precision improves the longer it operates; and (iii)

region-grid-based keying, which allows the same pipeline to scale from a single city to a national network simply by adding more region keys, without redesigning the fusion logic.

8. Summary

Overall, the design uses PySpark RDD transformations (map, filter, reduceByKey/groupByKey, join) to distribute and fuse three heterogeneous disaster-related data streams at scale, while a layered intelligent-agent architecture (Sensor Agents → Coordinator Agent → Response Agents) converts the fused, distributed computation into a confident, low-false-alarm early warning and coordinated response — satisfying the requirement for a scalable, fault-tolerant, real-time disaster detection system.